GLOBAL SOLUTION

Disciplina: Cloud

Trabalho: API

CyberIncident API

Plataforma de Gerenciamento de Incidentes de Segurança Cibernética

Aluno: Breno Aveiro

Data: 02/11/2026

Sumário

1. Introdução
2. Objetivos
3. Tecnologias
4. Arquitetura
5. Desenvolvimento da aplicação
6. SQLite
7. Azure
8. Implantação
9. Testes
10. Evidências
11. Segurança
12. Custos
13. Conclusão
14. Referências

1. Introdução

Incidentes de segurança cibernética — tentativas de acesso indevido, ataques de força bruta, mensagens de phishing, detecção de malware e possíveis vazamentos de dados — precisam ser registrados, classificados e acompanhados até a resolução. Em uma organização, essa tarefa é realizada por uma central de operações de segurança, conhecida pela sigla SOC (*Security Operations Center*).

Este trabalho apresenta a **CyberIncident API**, uma aplicação acadêmica que simula o núcleo de um sistema de registro de incidentes de um SOC. A aplicação não monitora redes nem detecta ameaças de verdade: ela permite cadastrar, consultar, classificar e acompanhar incidentes **fictícios**, com o objetivo de demonstrar o funcionamento de uma API REST hospedada em nuvem.

O projeto partiu de uma estrutura mais simples, uma API de tarefas, publicada anteriormente na mesma infraestrutura. A infraestrutura e o banco foram preservados, e o domínio da aplicação, o modelo de dados, os endpoints, a interface e a documentação foram substituídos pelo tema de segurança cibernética.

2. Objetivos

O objetivo geral é desenvolver, testar e publicar na Microsoft Azure uma API REST com persistência em SQLite, documentada e demonstrável pela internet. Os objetivos específicos são:

- modelar incidentes de segurança com categoria, severidade, status, origem e ativo afetado;
- oferecer operações completas de criação, consulta, atualização e exclusão, além de estatísticas calculadas a partir dos dados reais;
- construir um painel (dashboard) visual e uma documentação interativa com identidade própria;
- hospedar a aplicação em uma máquina virtual Azure, com Nginx, systemd e disco persistente;
- validar o funcionamento por testes automatizados locais e testes na URL pública;
- registrar evidências reais e documentar as decisões técnicas, os custos e as limitações.

3. Tecnologias

Tecnologia	Função no projeto
Python 3	Linguagem de programação da aplicação (versão 3.12 na VM Ubuntu).
FastAPI 0.115	Framework web que define os endpoints, valida dados e gera a documentação OpenAPI.
Pydantic 2	Validação dos dados de entrada e formato dos dados de saída (enums, tamanhos, endereço IP).
SQLAlchemy 2	Mapeamento objeto-relacional (ORM): converte objetos Python em comandos SQL parametrizados.
SQLite	Banco de dados relacional gravado em um único arquivo.
Uvicorn	Servidor ASGI que executa a aplicação FastAPI.

Nginx	Servidor web que recebe as requisições na porta 80 e as repassa à aplicação (proxy reverso).
systemd	Gerenciador de serviços do Linux: inicia a API com o sistema e a reinicia em caso de falha.
HTML, CSS e JavaScript	Dashboard e tema do Swagger, sem bibliotecas externas.
Swagger UI	Interface de documentação e teste dos endpoints, servida localmente.
Git e GitHub	Controle de versão e publicação do código.
Pytest e HTTPX	Testes automatizados.
Mermaid	Diagramas da arquitetura e dos fluxos.

4. Arquitetura

A requisição de um usuário percorre os seguintes componentes: navegador, IP público, grupo de segurança de rede (NSG), rede virtual, Nginx, Uvicorn/FastAPI e SQLite. O Diagrama 1 apresenta essa estrutura e o Diagrama 2 detalha o fluxo HTTP.

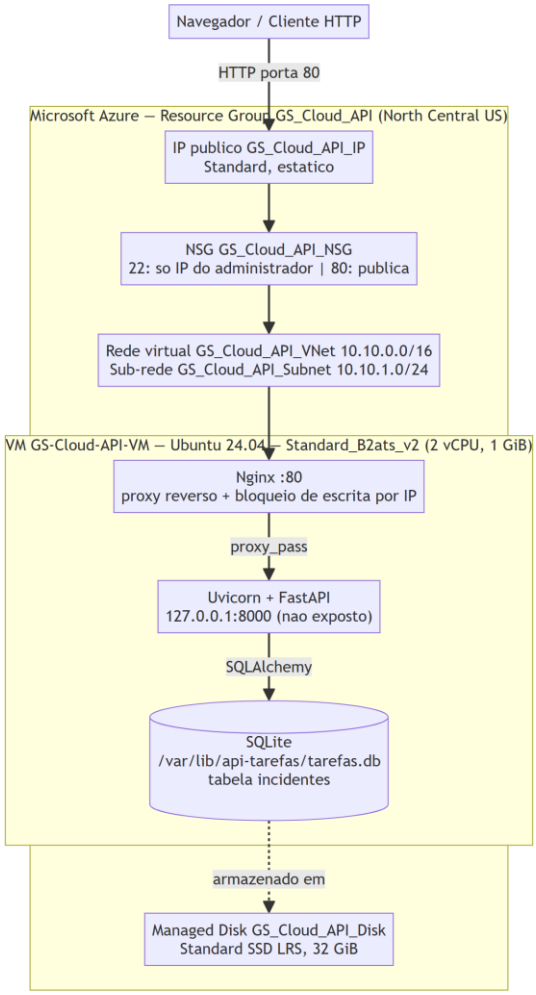


Diagrama 1 — Arquitetura da CyberIncident API na Microsoft Azure.

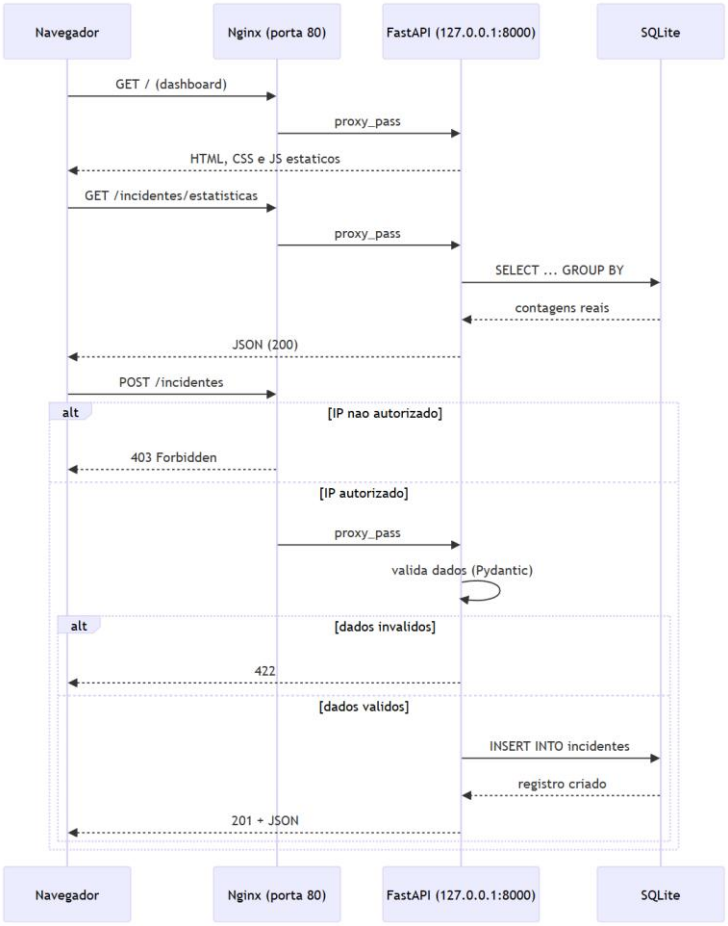


Diagrama 2 — Fluxo das requisições HTTP entre o navegador, o Nginx, o FastAPI e o SQLite.

O **Nginx** é o único componente que recebe tráfego da internet. Ele entrega as requisições ao Uvicorn, que escuta apenas em 127.0.0.1:8000 e, por isso, não é acessível de fora da VM. Essa separação reduz a superfície de ataque, pois o servidor da aplicação nunca fica exposto diretamente.

5. Desenvolvimento da aplicação

5.1 Modelo de dados

A tabela incidentes armazena os campos id, titulo, descricao, categoria, severidade, status, ip_origem, ativo_afetado, responsavel, criado_em, atualizado_em e resolvido_em. Os valores permitidos são:

- **severidade:** baixa, média, alta e crítica;
- **status:** aberto, em análise, contido e resolvido;
- **categoria:** phishing, malware, acesso indevido, força bruta, vazamento de dados, anomalia de rede e outros.

Os valores são garantidos em duas camadas: pelos tipos enumerados do Pydantic, na entrada da API, e por restrições CHECK no próprio banco de dados. As datas são gravadas em UTC.

5.2 Endpoints

Método e rota	Função	Códigos HTTP
GET /health	Verifica a API e a comunicação com o SQLite	200
GET /incidentes	Lista com paginação e filtros por severidade, status e categoria	200, 422
GET /incidentes/estatisticas	Contagens reais por status, severidade e categoria	200
GET /incidentes/{id}	Consulta um incidente	200, 404
POST /incidentes	Cadastra um incidente (status inicial: aberto)	201, 422
PUT /incidentes/{id}	Atualiza todos os campos	200, 404, 422
PATCH /incidentes/{id}/status	Altera somente o status	200, 404, 422
DELETE /incidentes/{id}	Exclui um incidente de demonstração	204, 404

A rota de estatísticas é declarada antes da rota de consulta por identificador, para não ser interpretada como um ID. Erros internos retornam HTTP 500 com mensagem genérica, sem expor detalhes técnicos.

5.3 Validações e fluxo de status

Os dados de entrada são validados antes de chegar ao banco: título e descrição com tamanhos mínimo e máximo, categoria e severidade restritas às listas permitidas e endereço IP verificado com o módulo ipaddress (aceita IPv4 e IPv6). Uma string vazia em campo opcional é tratada como ausente.

O status segue um fluxo de tratamento, representado no Diagrama 4: aberto, em análise, contido e resolvido. Mudanças fora do fluxo, como passar de *aberto* direto para *resolvido*, são rejeitadas com HTTP 422. Ao resolver um incidente, o campo resolvido_em recebe a data e hora UTC; ao reabri-lo, o campo é limpo.

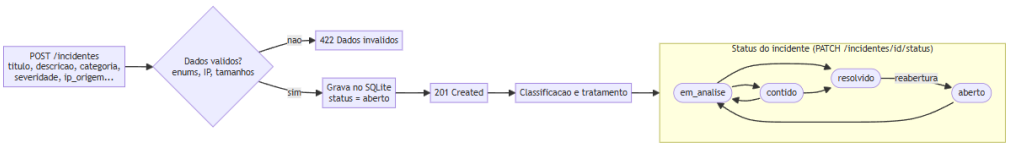


Diagrama 4 — Cadastro e classificação de incidentes e fluxo de status.

5.4 Dashboard e Swagger

A rota GET / entrega um painel com tema escuro, de estilo SOC. Ele exibe o indicador de disponibilidade da API, quatro cartões (total, críticos, abertos e resolvidos), um gráfico de barras por severidade e a tabela dos incidentes mais recentes. **Todos os números vêm do SQLite**, por meio dos endpoints de estatísticas e de listagem. O painel é feito apenas com HTML, CSS e JavaScript, servidos pelo próprio FastAPI, sem um segundo servidor e com baixo consumo de memória. Os dados são inseridos na página com textContent, nunca como HTML, o que impede a execução de código injetado (XSS); além disso, uma política de segurança de conteúdo (CSP) restringe os recursos carregados.

A documentação em /docs usa o Swagger UI com os arquivos servidos localmente (sem CDN externo), tema escuro, faixa de marca, descrições em português e seções separadas por tipo de operação. Continua totalmente interativa e o arquivo /openapi.json permanece disponível.

6. SQLite

O SQLite é um banco de dados relacional que guarda tudo — tabelas, índices e dados — em um único arquivo (/var/lib/api-tarefas/tarefas.db). Ele não exige um servidor de banco de dados separado, não gera custo adicional de infraestrutura e atende ao volume de dados e ao número de acessos deste trabalho. O modo WAL (*write-ahead logging*) está ativo, o que reduz bloqueios entre leituras e escritas.

O SQLite tem limites: não possui replicação e lida mal com muitas escritas simultâneas. Por isso não é a escolha para sistemas de grande escala, em que se usaria um serviço gerenciado como o Azure SQL Database. O Diagrama 3 mostra a estrutura da tabela.

INCIDENTES			
INTEGER	id	PK	
VARCHAR	titulo		
TEXT	descricao		
VARCHAR	categoria		phishing, malware, acesso_indevido, forca_bruta, vazamento_dados, anomalia_rede, outros
VARCHAR	severidade		baixa, media, alta, critica
VARCHAR	status		aberto, em_analise, contido, resolvido
VARCHAR	ip_origem		opcional, IPv4 ou IPv6
VARCHAR	ativo_afetado		
VARCHAR	responsavel		opcional
DATETIME	criado_em		UTC
DATETIME	atualizado_em		UTC
DATETIME	resolvido_em		preenchido ao resolver

Diagrama 3 — Modelo de dados da tabela incidentes.

Migração. A tabela antiga tarefas foi mantida no mesmo arquivo. Ao iniciar, a aplicação executa create_all, que cria somente as tabelas ausentes, de forma repetível e sem apagar dados. Antes da atualização foi feito um backup consistente com a API de backup do SQLite, e sua integridade foi verificada.

7. Azure

Todos os recursos pertencem ao grupo de recursos GS_Cloud_API, na região North Central US.

Recurso	Valor real	Por que existe
Resource Group	GS_Cloud_API	Agrupa os recursos; apagar o grupo remove tudo e encerra a cobrança.
Máquina virtual	GS-Cloud-API-VM, Ubuntu Server 24.04 LTS	Computador virtual Linux que executa a aplicação.
SKU	Standard_B2ats_v2: 2 vCPUs e 1 GiB de RAM	Menor tamanho permitido e com cota na assinatura Azure for Students; do tipo *burstable*, acumula crédito de CPU para picos.
Disco gerenciado	GS_Cloud_API_Disk, Standard SSD LRS, 32 GiB	Armazenamento persistente: o SQLite sobrevive a reinicializações. LRS mantém três cópias no mesmo datacenter.
Rede virtual	GS_Cloud_API_VNet (10.10.0.0/16) e sub-rede GS_Cloud_API_Subnet (10.10.1.0/24)	Rede privada isolada em que a VM recebe o IP 10.10.1.4 por meio de uma interface de rede (NIC).
IP público	GS_Cloud_API_IP, SKU Standard, estático	Endereço fixo para acessar a API pela internet.
NSG	GS_Cloud_API_NSG	Firewall: SSH (22) somente do IP do administrador; HTTP (80) público; demais portas negadas.

Por que a SKU Standard_B1s não foi usada. A primeira opção planejada era a Standard_B1s, mas ela não está disponível na assinatura utilizada. Uma política também limita as regiões em que é possível criar recursos, e as máquinas liberadas em outras regiões tinham cota zero. A Standard_B2ats_v2 em North Central US foi a alternativa de menor custo permitida.

As Figuras 1 a 6 apresentam os recursos no Portal Azure.

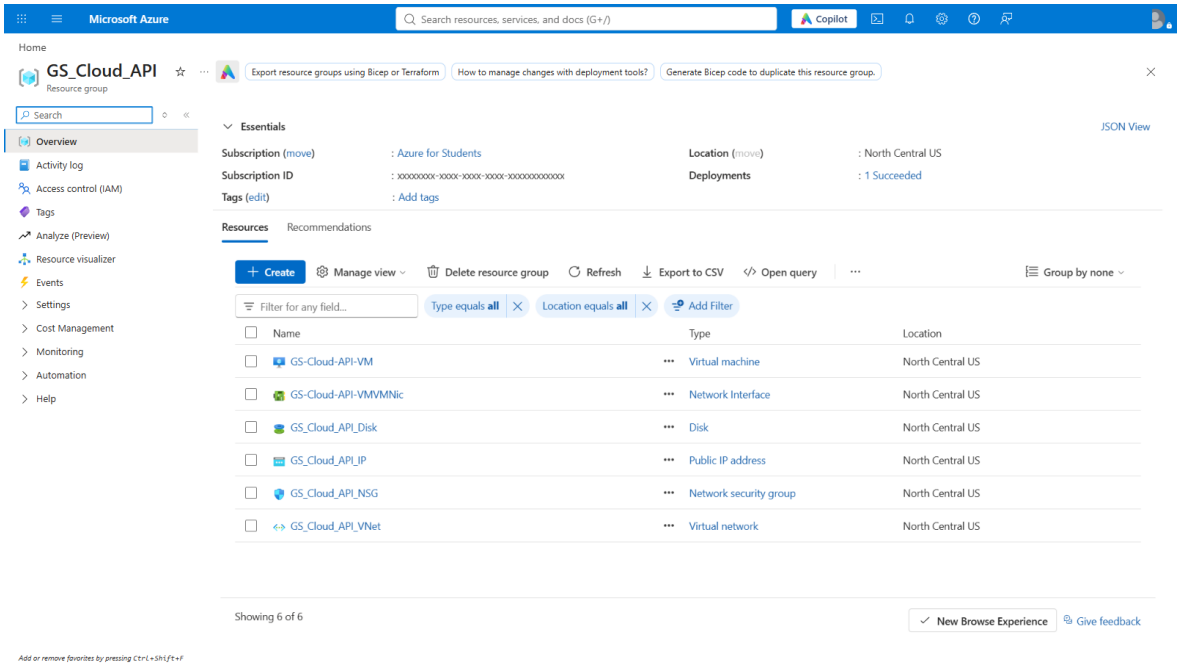


Figura 1 — Grupo de recursos utilizado para organizar os componentes da aplicação hospedada na Microsoft Azure.

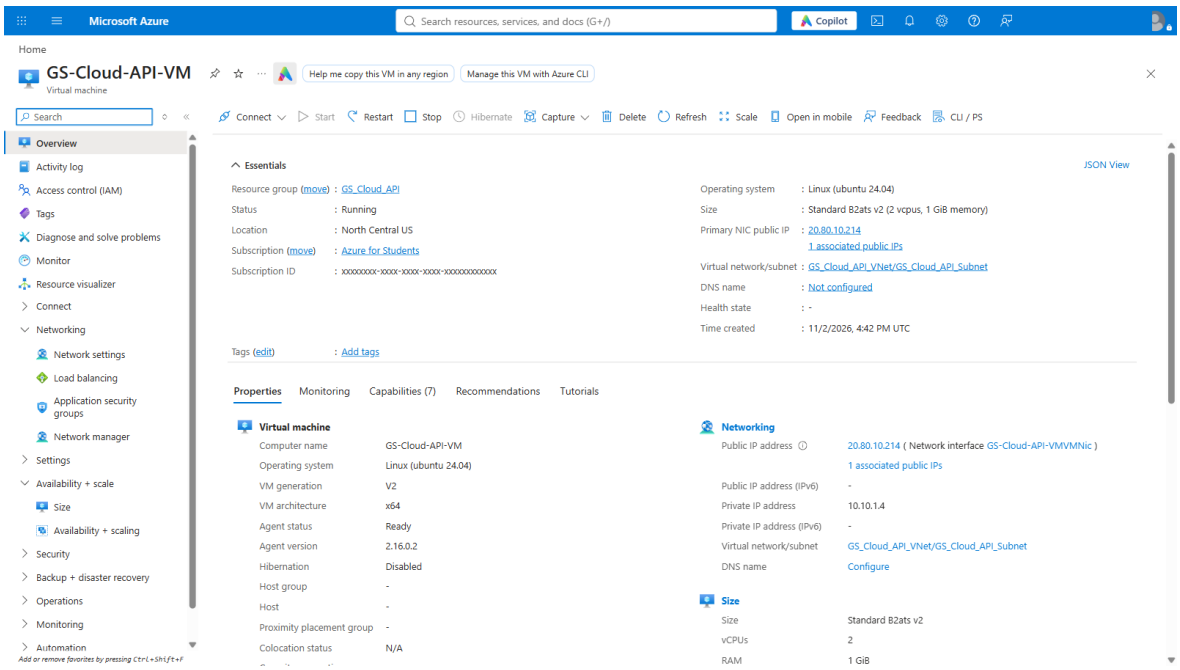


Figura 2 — Máquina virtual utilizada para executar a API REST na Microsoft Azure.

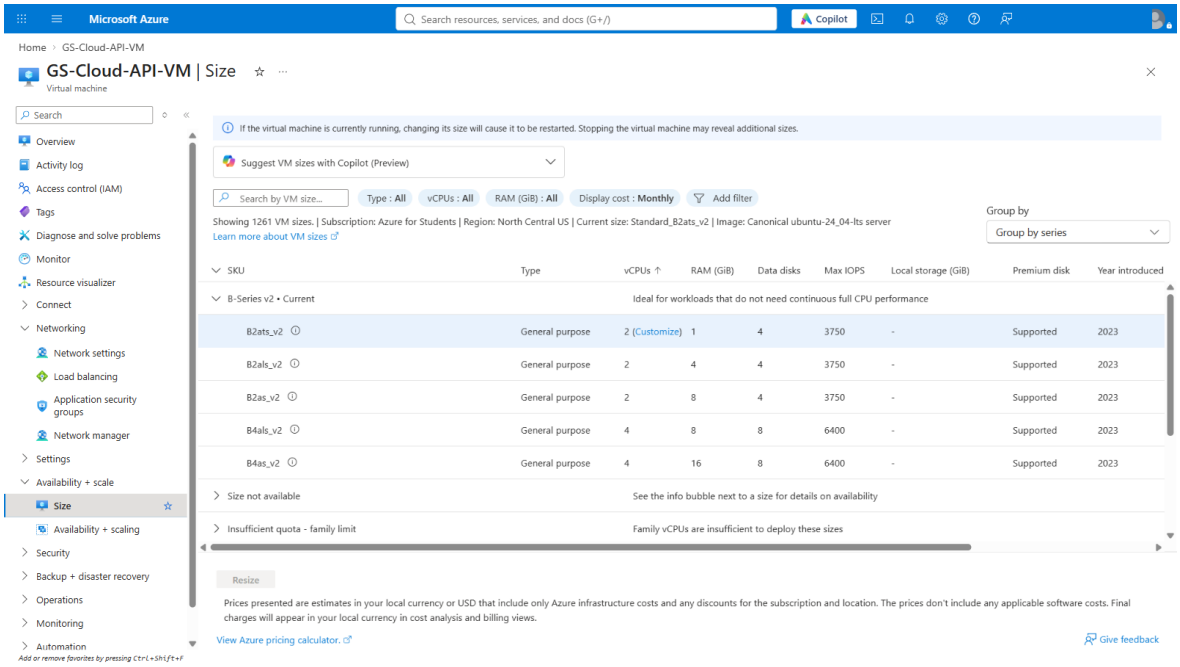


Figura 3 — Configuração de processamento e memória da máquina virtual utilizada no projeto.

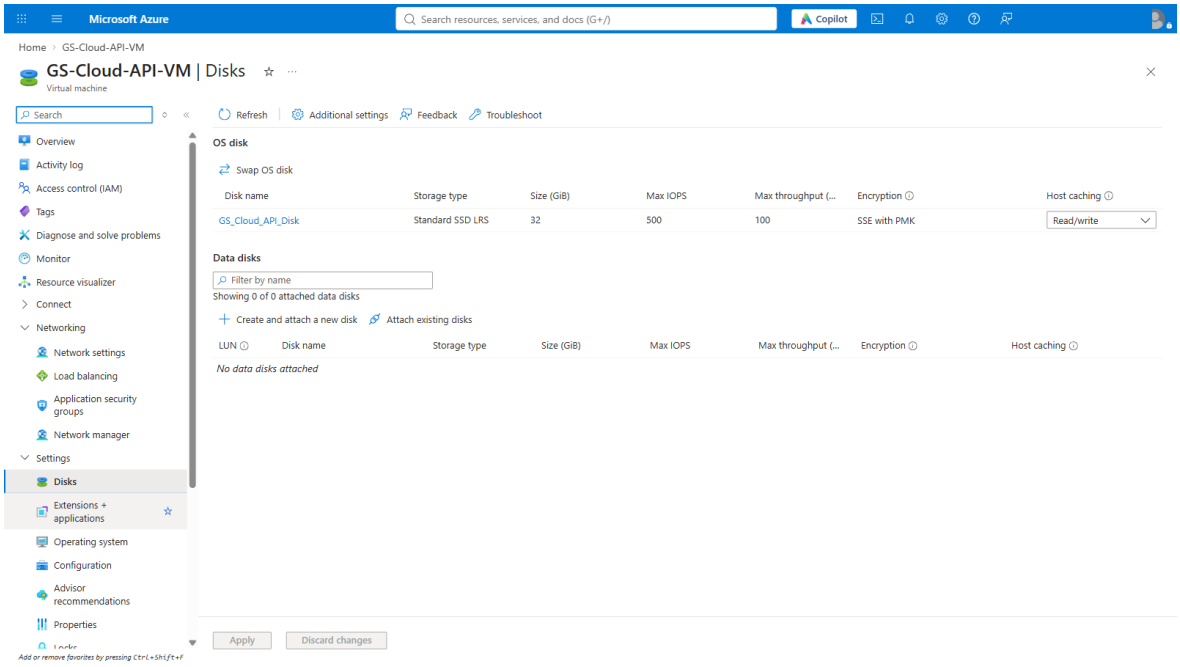


Figura 4 — Armazenamento persistente disponibilizado por meio dos discos gerenciados da Microsoft Azure.

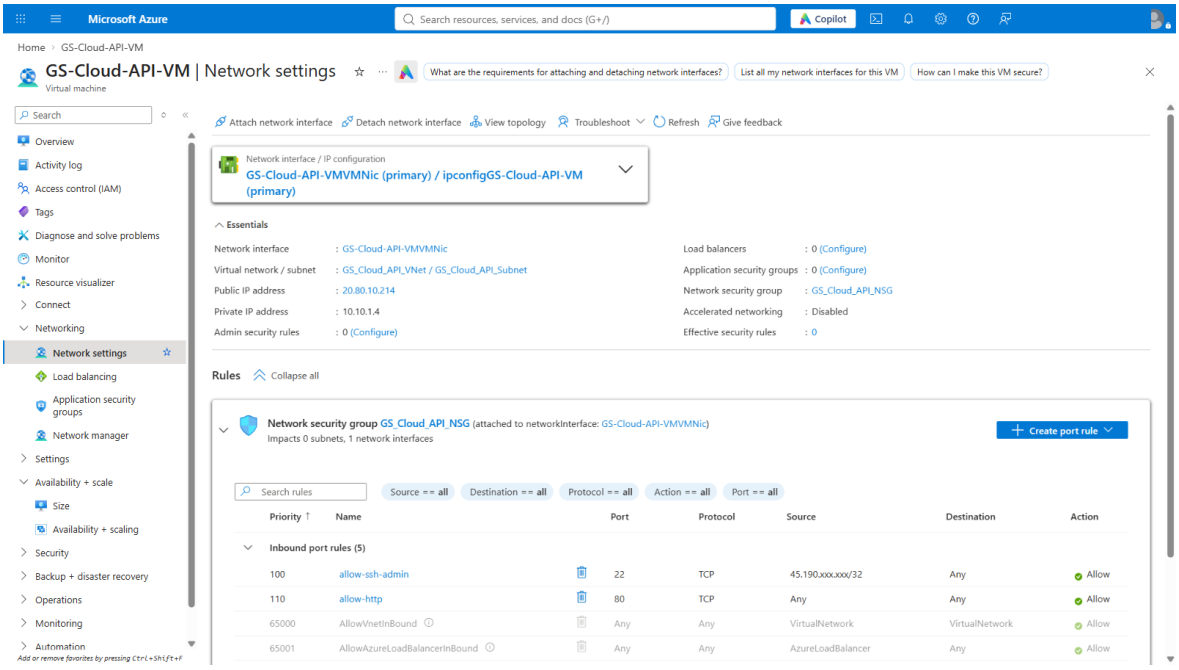


Figura 5 — Infraestrutura de rede responsável pela comunicação da máquina virtual com os demais componentes.

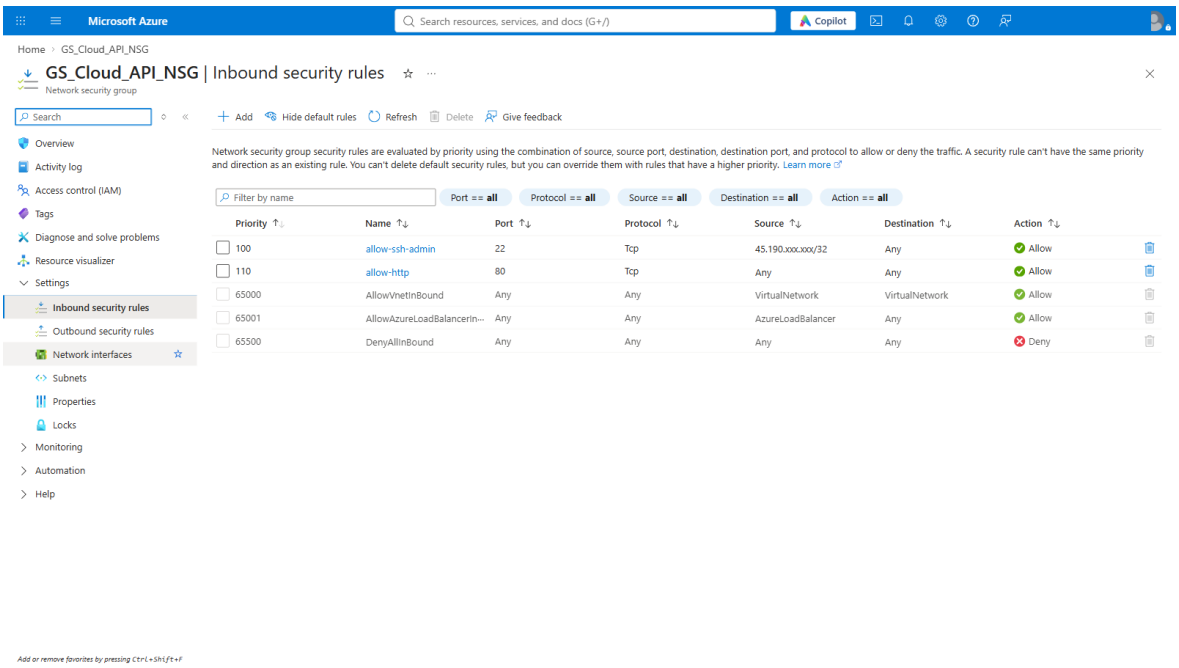


Figura 6 — Regras de segurança de rede utilizadas para controlar o acesso à máquina virtual.

8. Implantação

O código-fonte é versionado no GitHub. Na VM, o script deploy/install.sh instalou o Python, o Nginx e as dependências, criou o usuário de serviço sem privilégios apitarefas, o ambiente virtual Python, o serviço systemd e a configuração do Nginx. As atualizações posteriores usam deploy/deploy.sh, que: (1) faz backup do SQLite e **aborta se o backup falhar**; (2) atualiza o código com git pull; (3) instala dependências; (4) atualiza a unidade systemd; (5) reinicia o serviço; e (6) valida o /health.

O serviço api-tarefas (o nome foi mantido da versão anterior para preservar a configuração) executa o Uvicorn como usuário sem privilégios, com isolamento do systemd (ProtectSystem=strict, NoNewPrivileges) e permissão de escrita apenas em /var/lib/api-tarefas. O arquivo do banco tem permissão 600.

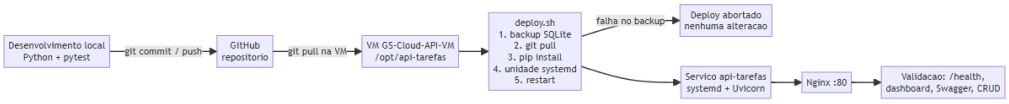


Diagrama 5 — Fluxo de implantação do GitHub para a Azure.

9. Testes

Testes automatizados locais. A suíte reúne 55 testes aprovados: 40 da CyberIncident API e 15 do instalador. Eles usam um banco SQLite temporário e isolado e cobrem inicialização, health check, CRUD, paginação e filtros, fluxo e datas de status, validações (severidade, status, categoria, IP e campos obrigatórios), estatísticas com banco vazio e preenchido, persistência após reabrir o banco, preservação da tabela antiga, OpenAPI, dashboard e arquivos estáticos, ausência de CDN, XSS, injeção de SQL, erro interno sem vazamento de detalhes, CORS fechado e as proteções do Nginx e do systemd. Há um aviso de depreciação de uma dependência, exibido na Figura 18.

Testes na URL pública. O script scripts/testar_producao.sh executou 30 verificações com 0 falhas: disponibilidade e interface, coerência das estatísticas com a listagem, CRUD completo, transições de status, validações (422) e recurso inexistente (404), persistência após reiniciar o serviço, porta 8000 fechada, bloqueio de escrita para IP não autorizado (HTTP 403 nos quatro métodos) e preservação da tabela antiga.

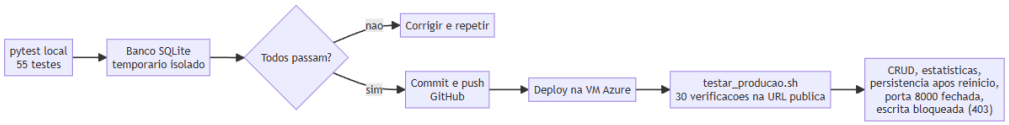


Diagrama 6 — Fluxo de testes e validação.

10. Evidências

As figuras a seguir foram obtidas da aplicação e da infraestrutura reais. As Figuras 7 a 13 e 17 são capturas de páginas abertas no navegador (a faixa escura superior, com a URL e o

horário, foi acrescentada na captura). As Figuras 14, 15, 16 e 18 são janelas reais de console. O endereço IP do administrador foi mascarado.

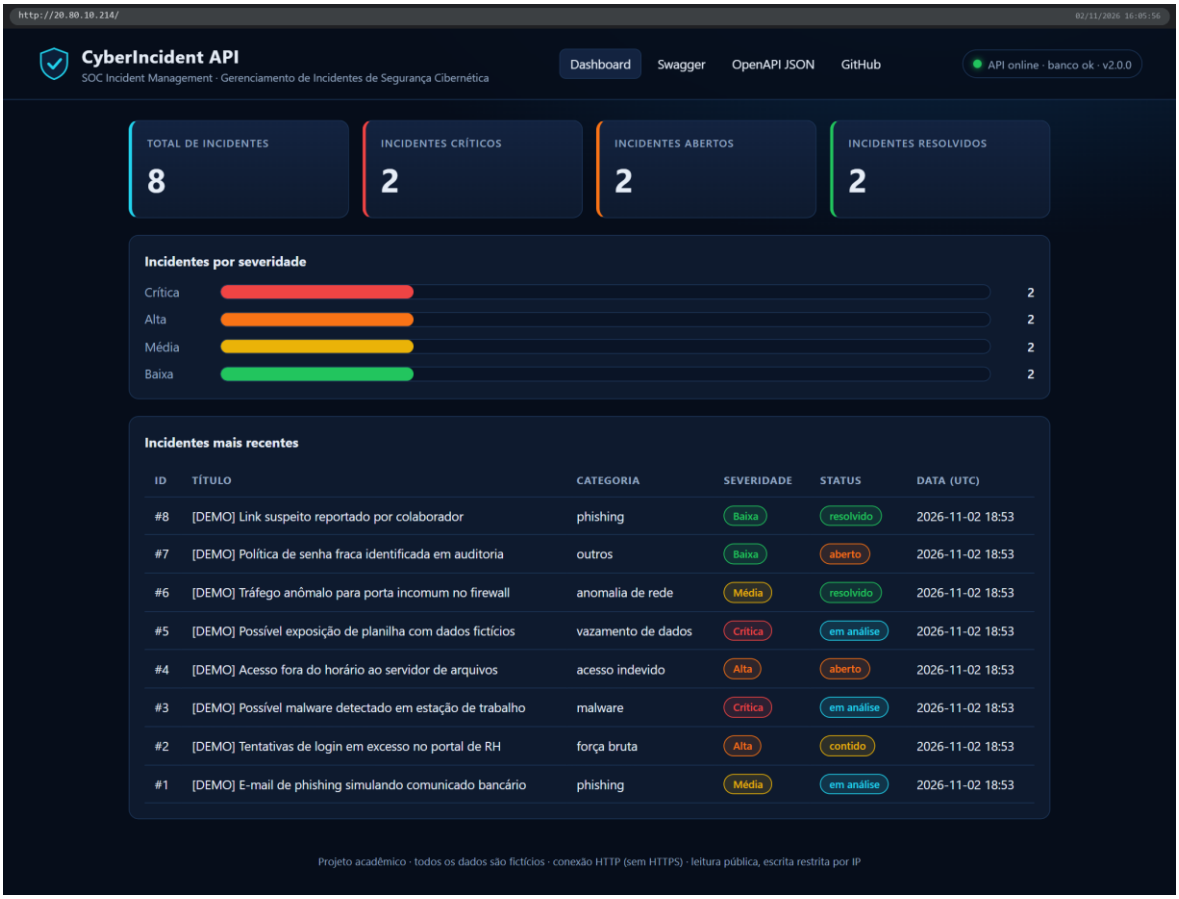


Figura 7 — Dashboard da CyberIncident API com indicadores, gráfico por severidade e tabela de incidentes.

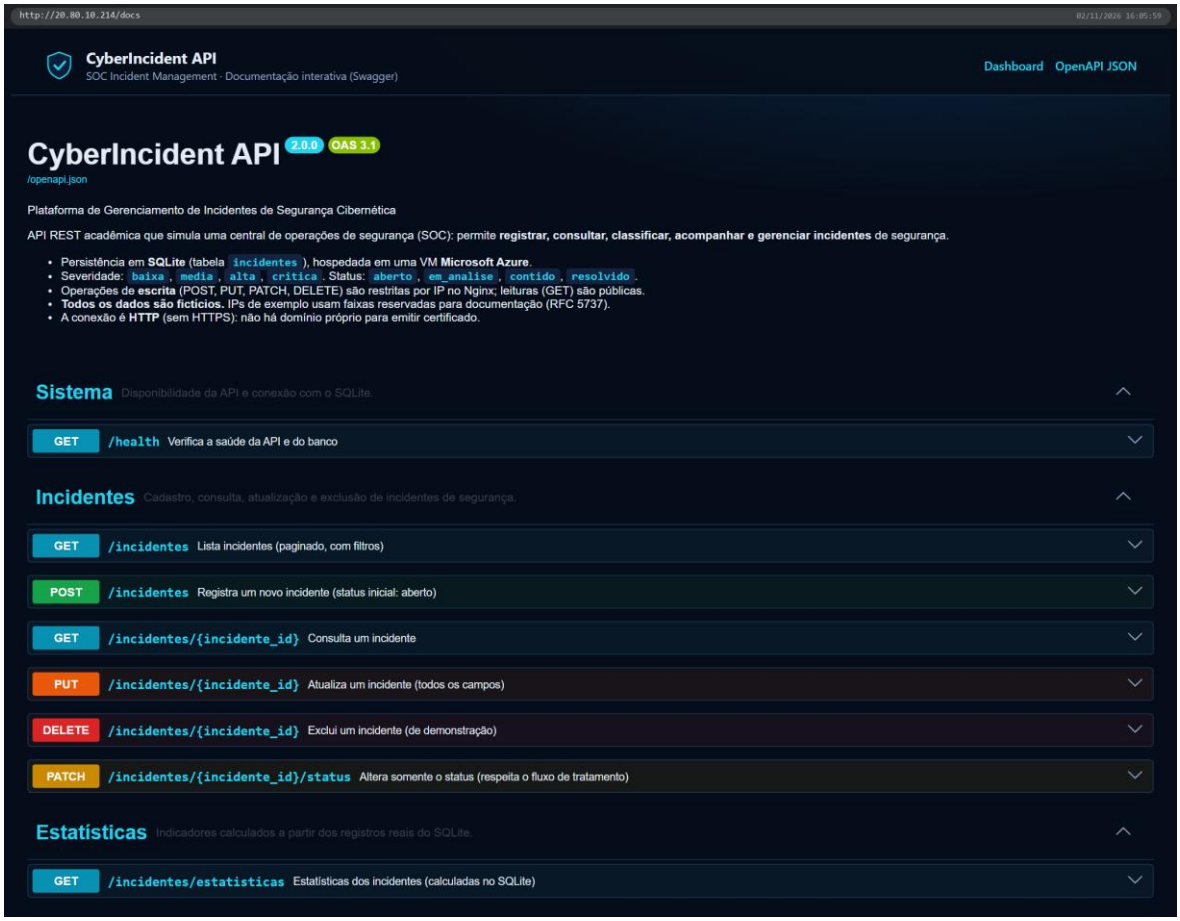


Figura 8 — Documentação Swagger/OpenAPI personalizada da aplicação publicada na Microsoft Azure.

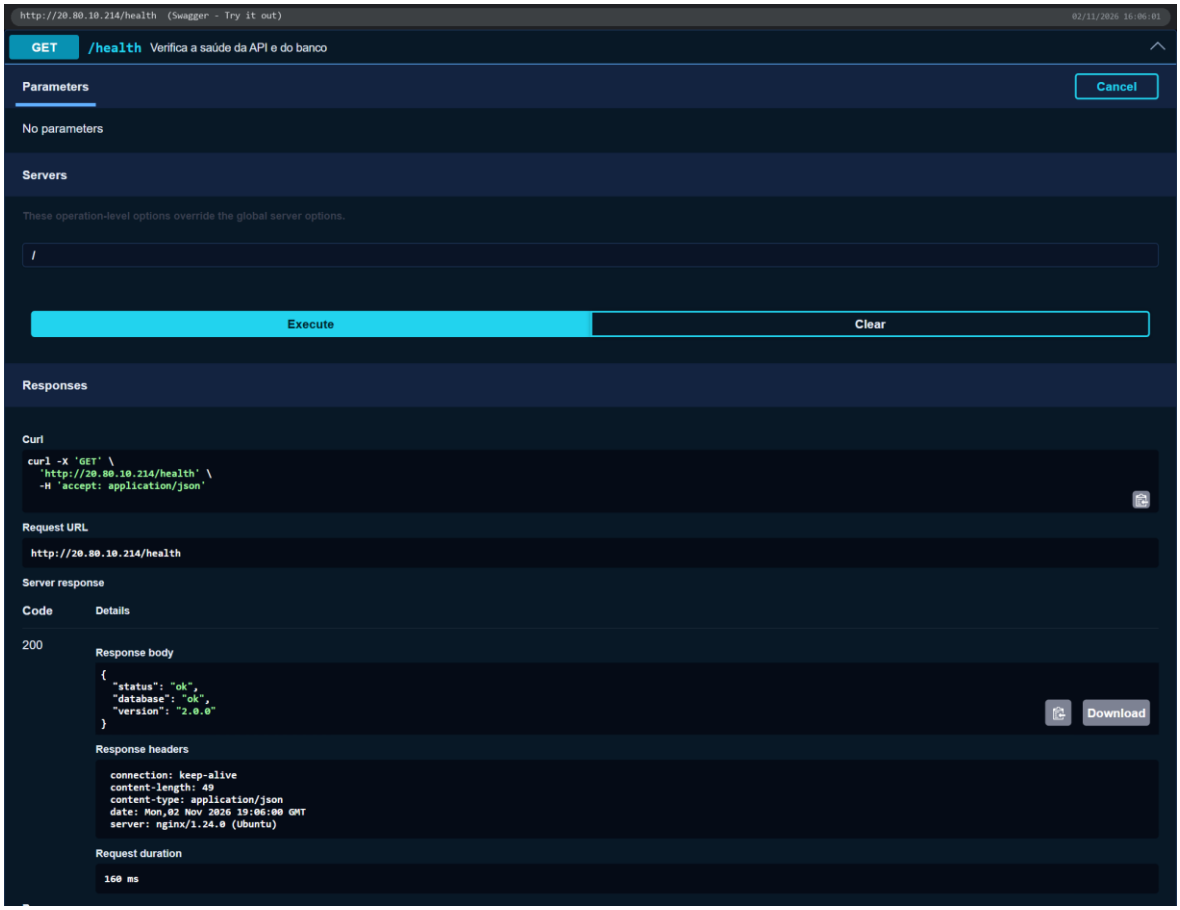


Figura 9 — Verificação da disponibilidade da API e da conexão com o banco SQLite (HTTP 200).

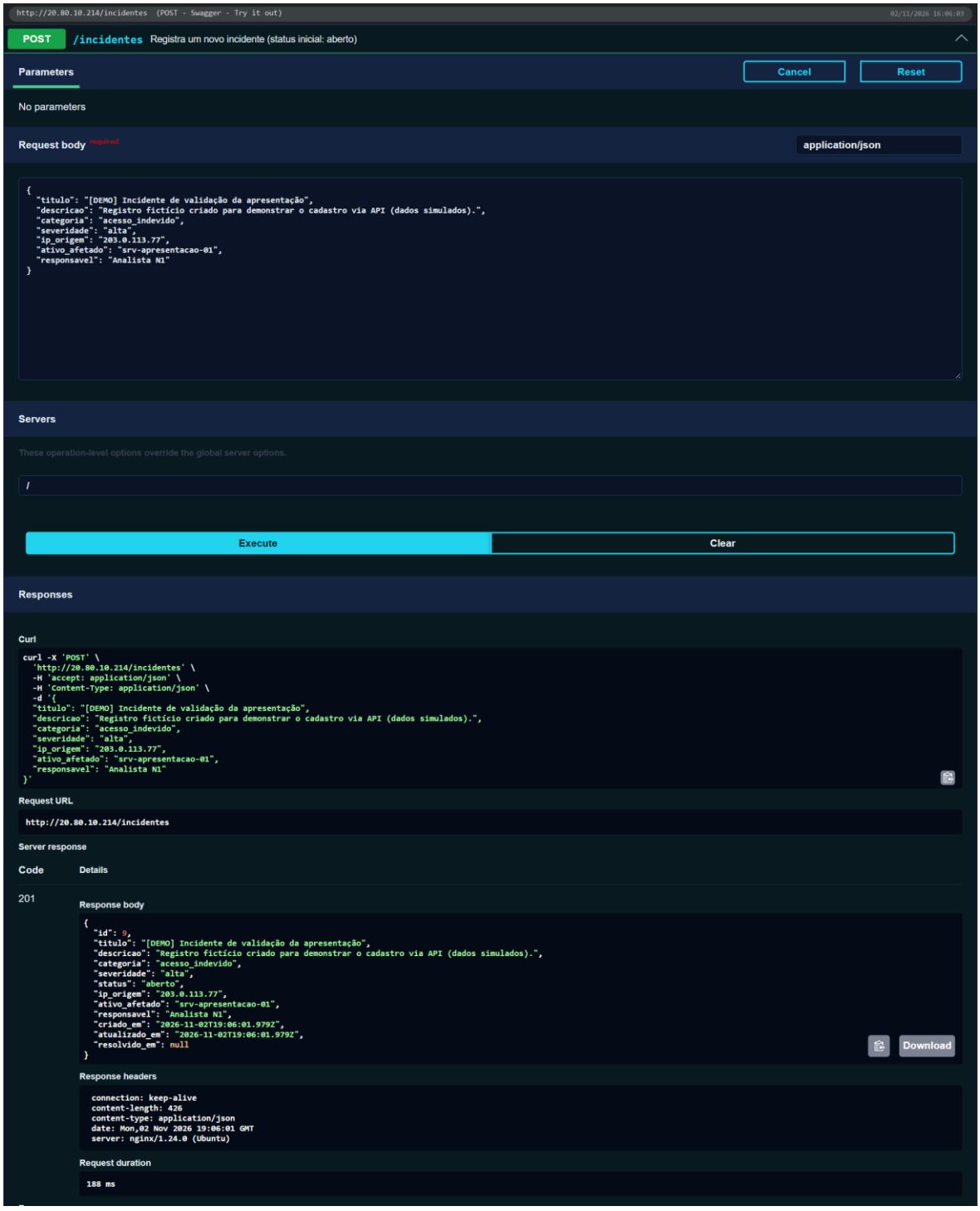


Figura 10 — Cadastro de incidente por meio de requisição HTTP POST (HTTP 201).

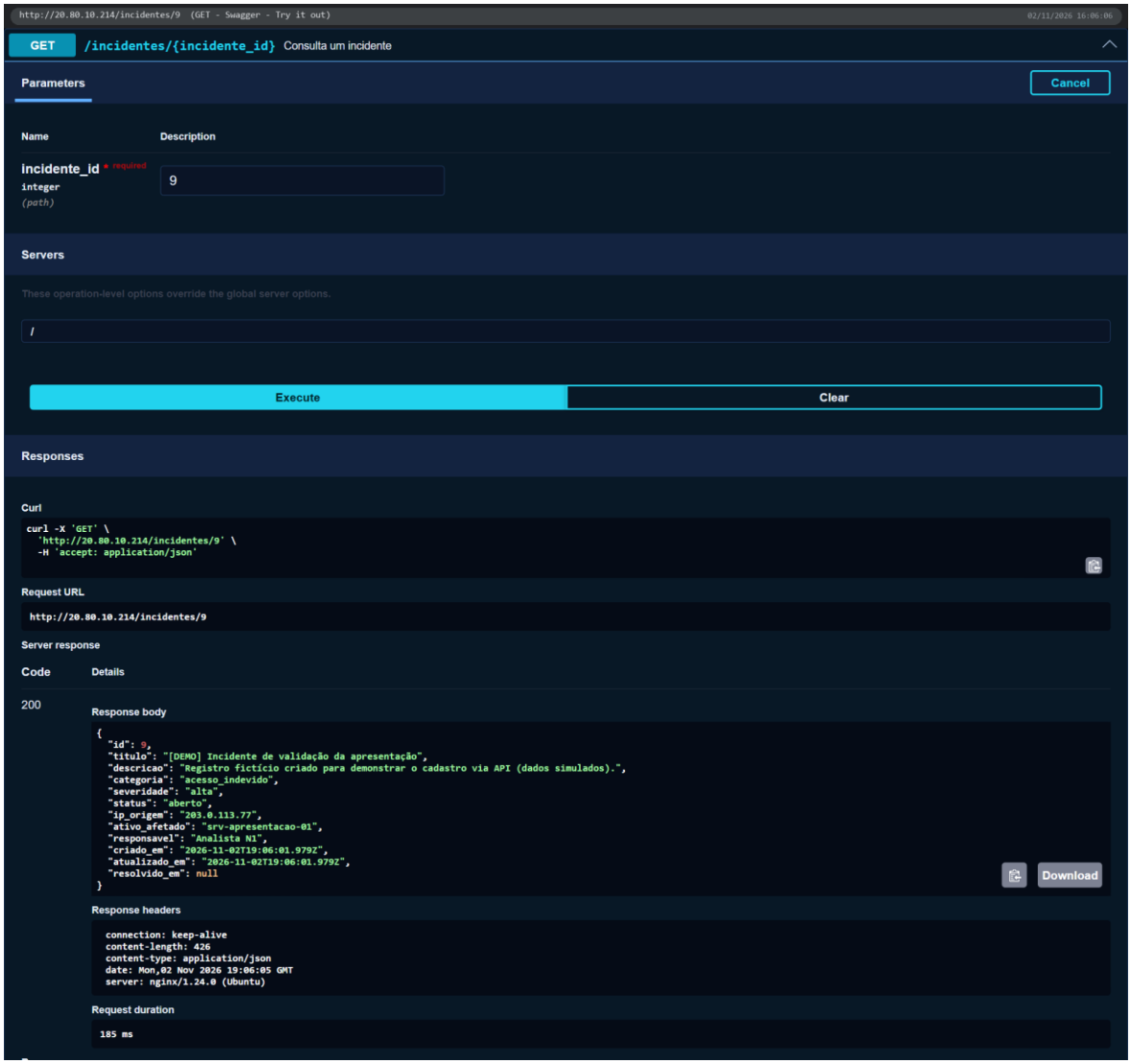


Figura 11 — Consulta de incidente persistido no SQLite por meio de requisição HTTP GET (HTTP 200).

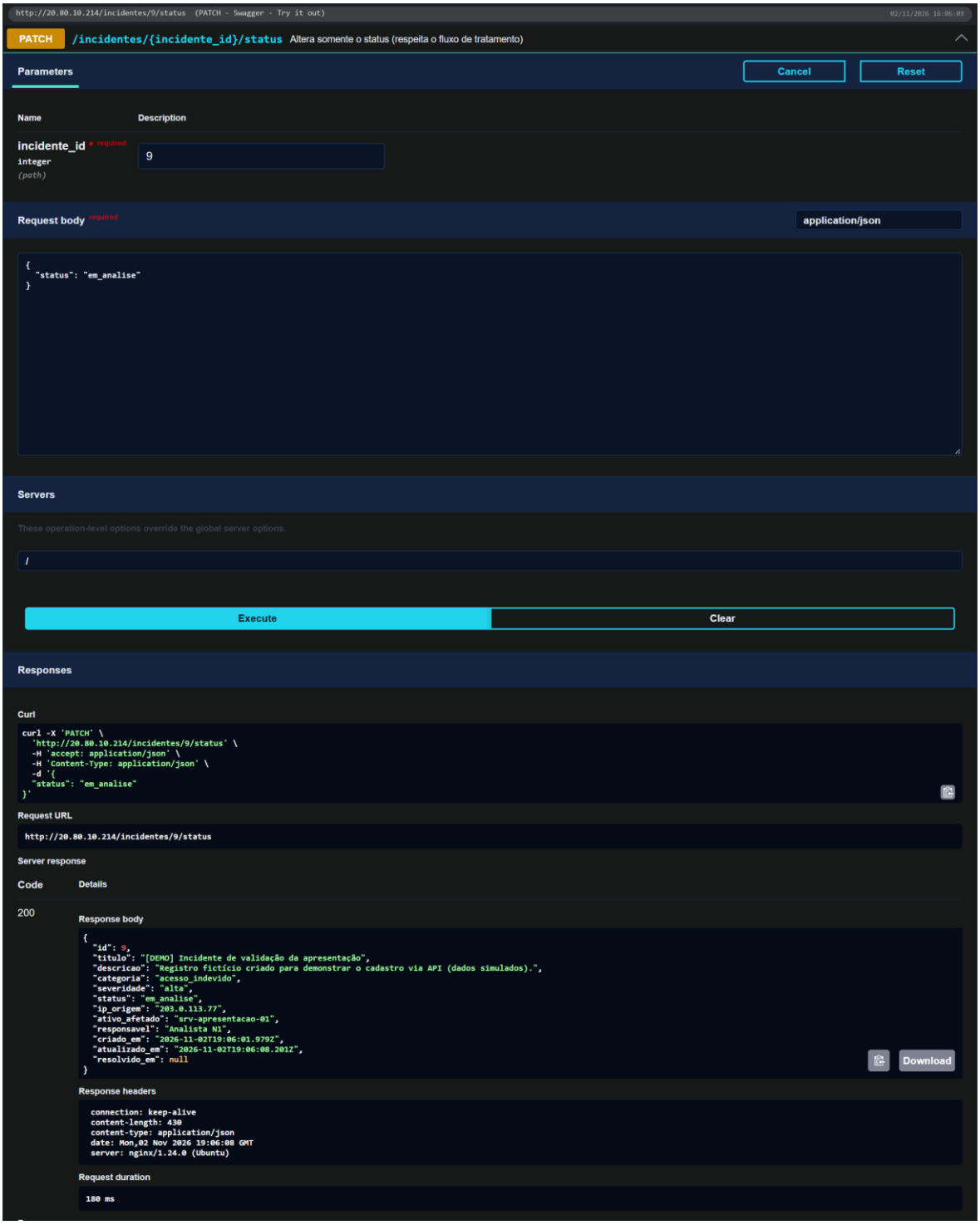


Figura 12 — Alteração de status do incidente por meio de requisição HTTP PATCH.

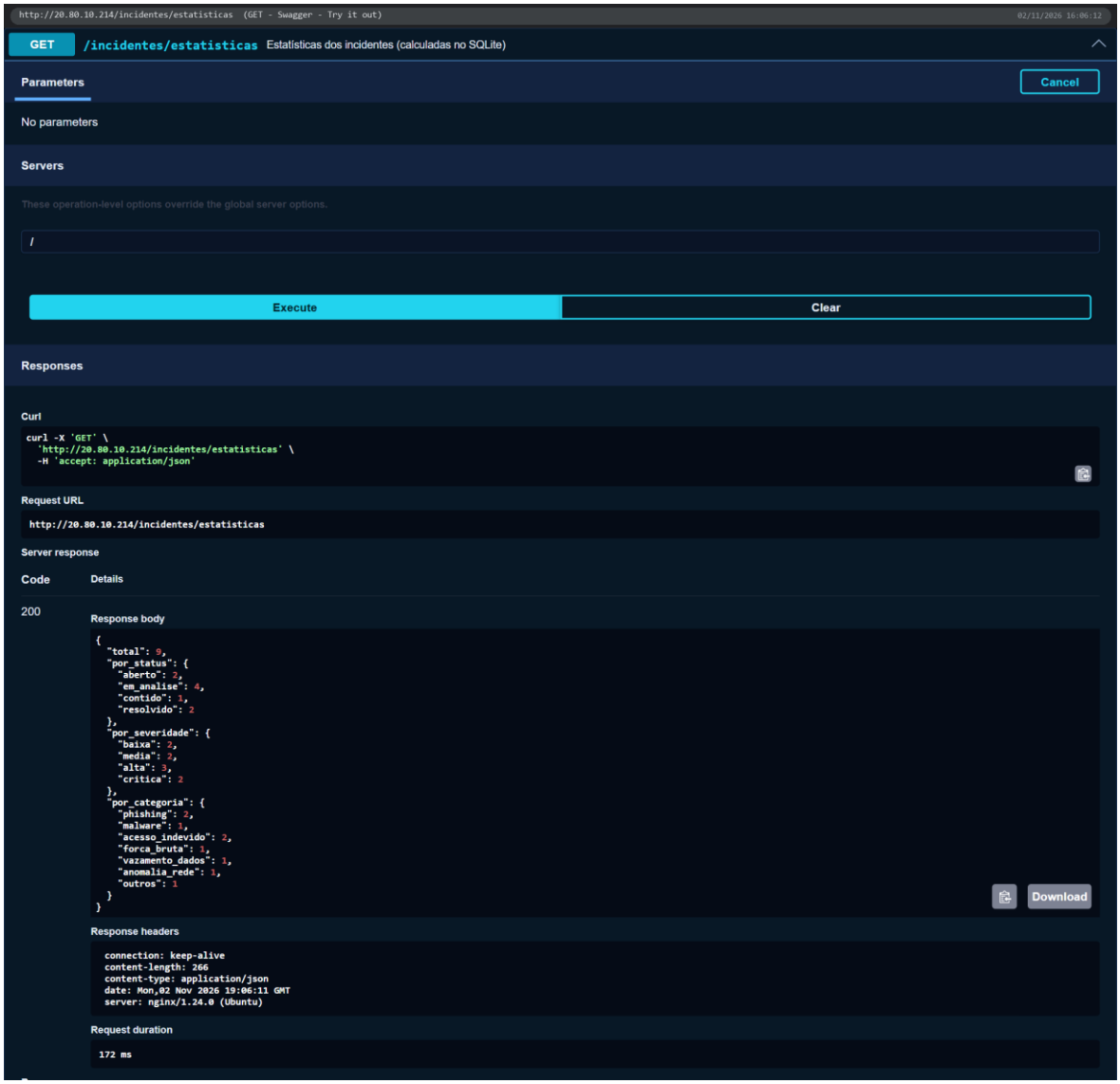


Figura 13 — Estatísticas calculadas a partir dos registros do SQLite.

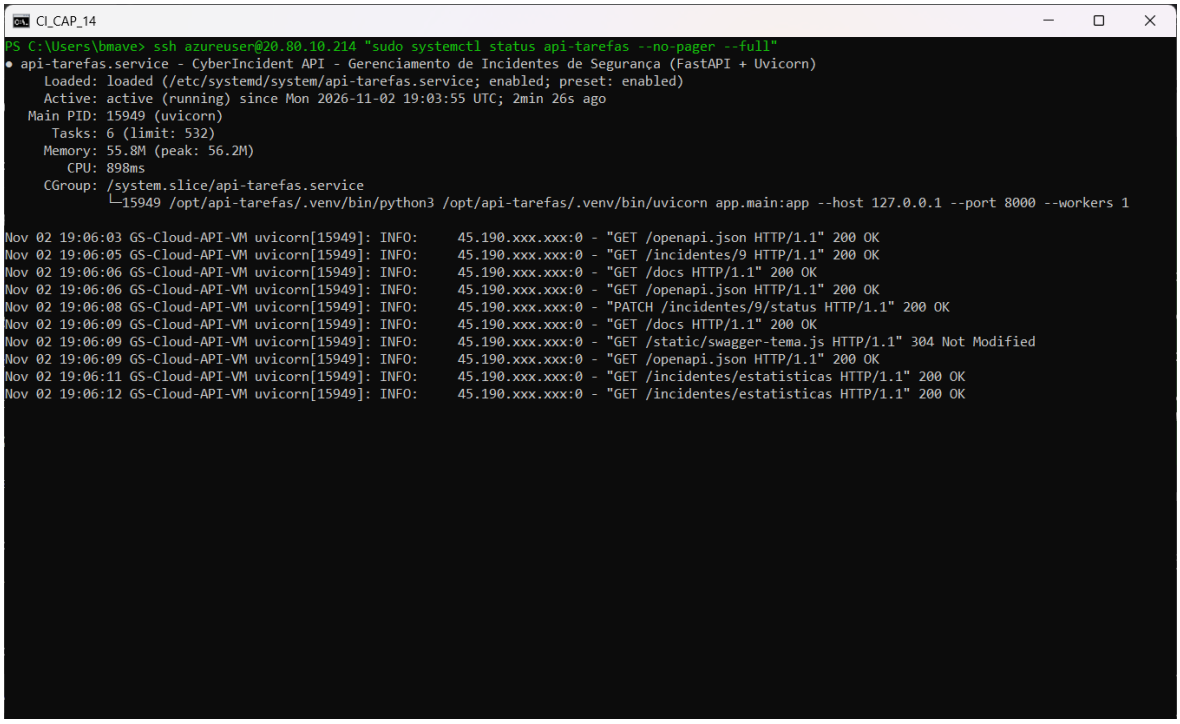


Figura 14 — Serviço systemd responsável pela execução contínua da API na máquina virtual Linux.

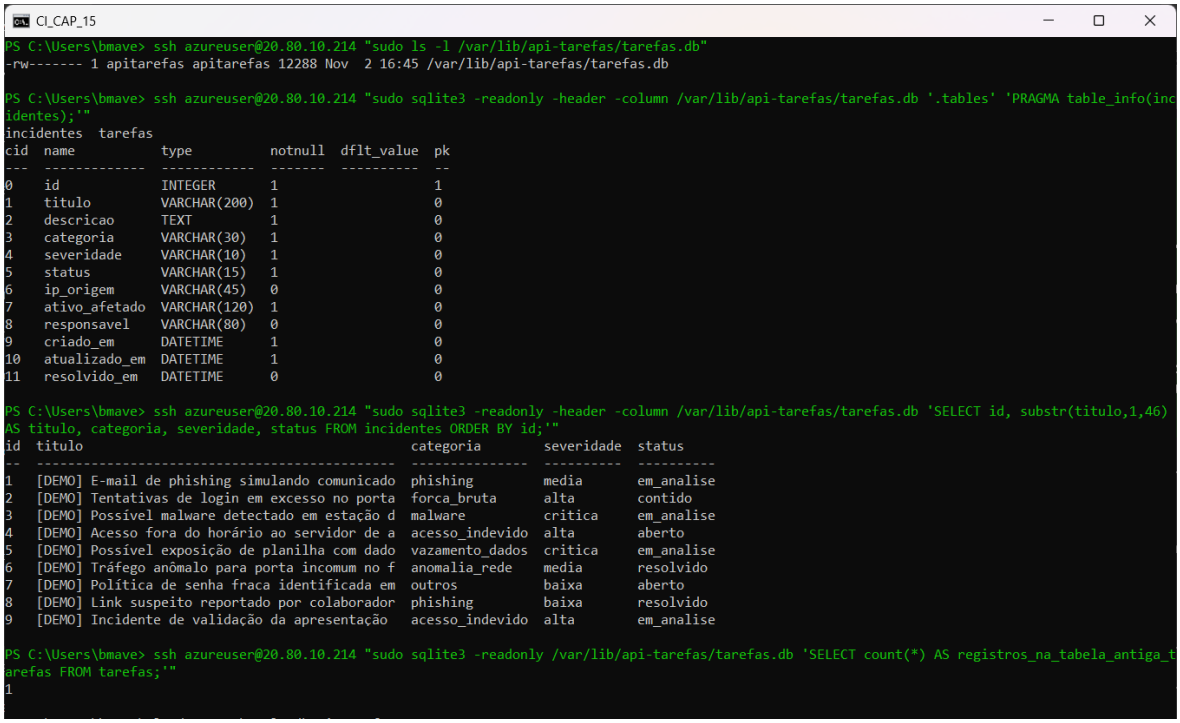


Figura 15 — Consulta direta ao banco SQLite armazenado no disco persistente da máquina virtual.

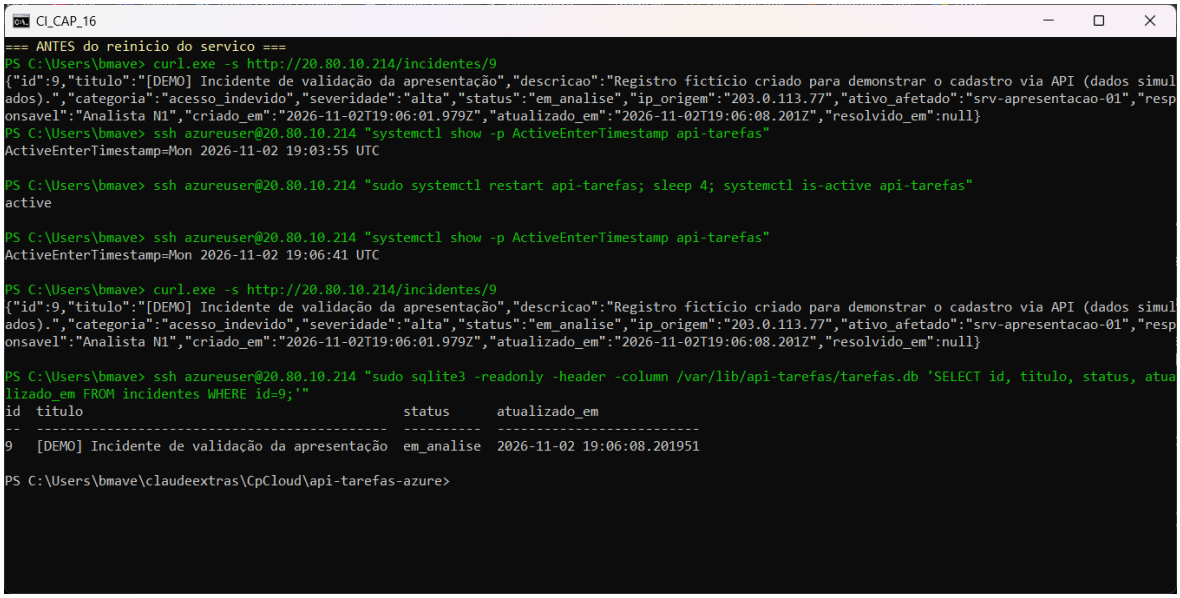


Figura 16 — Persistência dos registros SQLite após a reinicialização do serviço da aplicação.

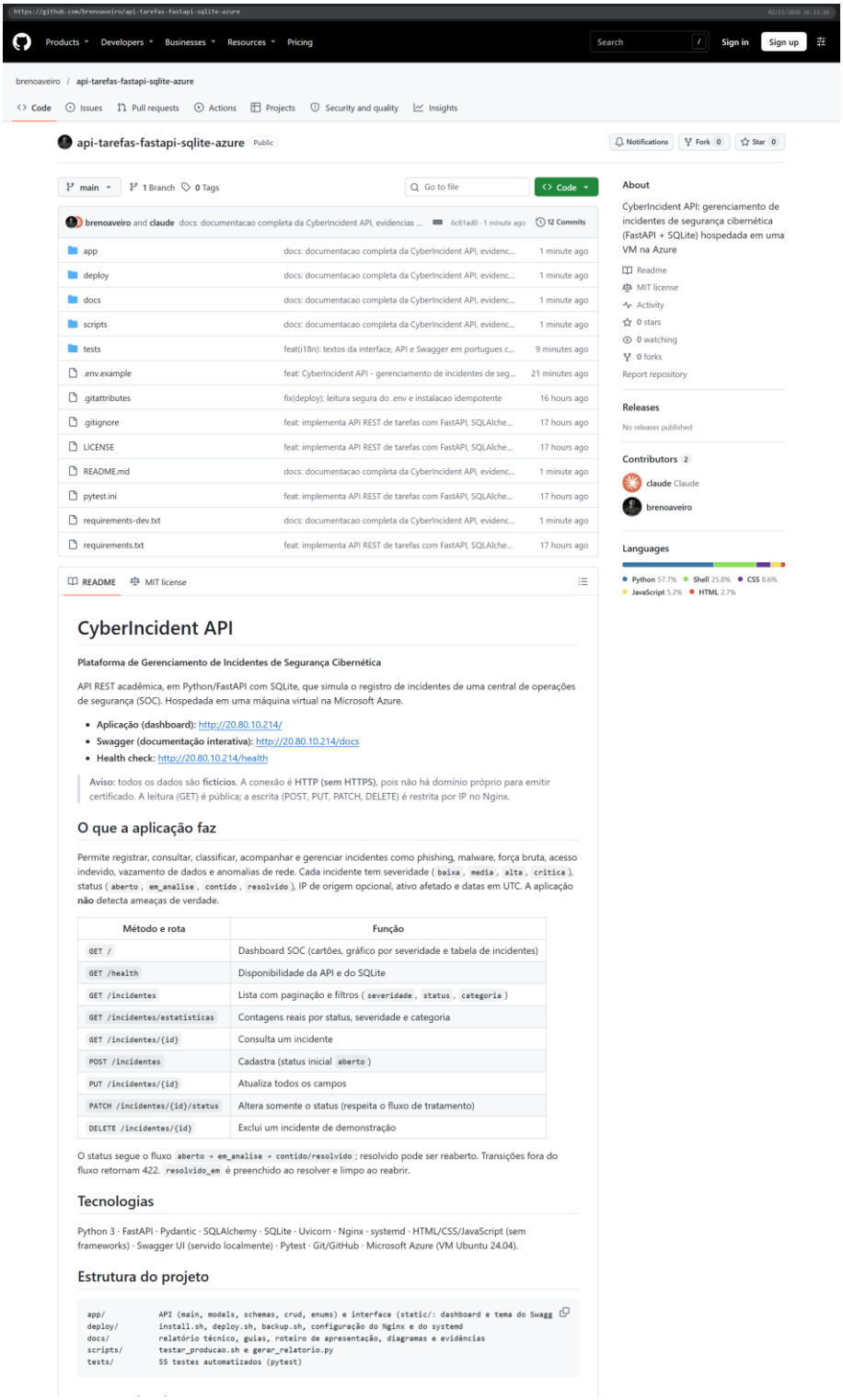


Figura 17 — Repositório GitHub utilizado para armazenar e versionar o código-fonte da aplicação.

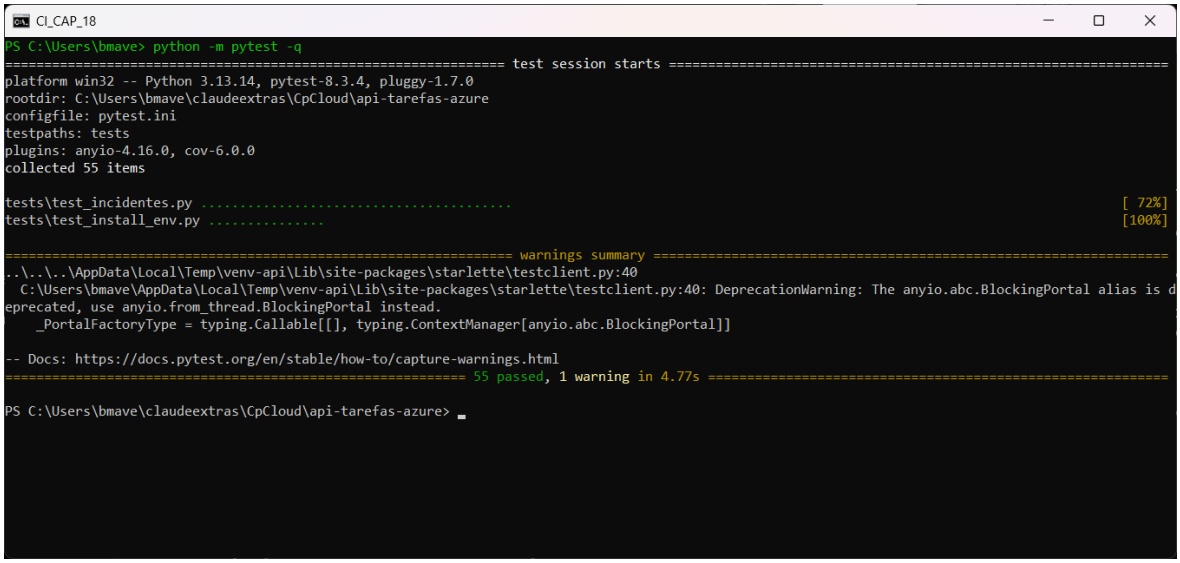


Figura 18 — Execução dos testes automatizados utilizados para validar o funcionamento da API.

11. Segurança

Controles aplicados.

- O NSG permite SSH somente a partir do IP do administrador e libera apenas a porta 80 ao público. A porta 8000 do Uvicorn nunca é exposta.
- A API não possui autenticação. Para compensar, o Nginx libera leitura (GET) ao público e permite escrita (POST, PUT, PATCH e DELETE) apenas a IPs autorizados, que ficam em um arquivo local da VM, fora do GitHub. Os demais recebem HTTP 403.
- O acesso à VM é por chave SSH, sem senha. O arquivo .env e o banco têm permissão restrita e não são publicados.
- Todas as consultas usam o ORM com parâmetros, o que impede a injeção de SQL. O painel não interpreta dados como HTML, e há uma política de segurança de conteúdo. Não há CORS aberto.
- Os dados são fictícios e os IPs de exemplo usam as faixas reservadas para documentação (RFC 5737).

Limitações. A conexão usa **HTTP, sem HTTPS**, portanto o tráfego não é criptografado. Não há domínio próprio para emitir um certificado válido; com um domínio, seria possível usar o Let's Encrypt. Também não há autenticação de usuários, o SQLite não é indicado para alta concorrência e a VM tem apenas 1 GiB de memória (foi criado um arquivo de swap de 1 GiB). Como melhorias, seriam necessários domínio e HTTPS, autenticação com tokens, um banco gerenciado e monitoramento.

12. Custos

Os recursos que geram cobrança são a máquina virtual, o disco gerenciado e o IP público. A estimativa abaixo usa os preços de tabela consultados na API pública de preços da Azure durante o projeto e considera a VM ligada 24 horas por dia, em Linux e sob demanda. Os valores variam por região e devem ser confirmados na calculadora oficial antes de qualquer decisão.

Recurso	Preço de tabela	Estimativa mensal

VM Standard_B2ats_v2 (North Central US)	US$ 0,0094 por hora	cerca de US$ 6,9
Disco Standard SSD de 32 GiB	cerca de US$ 2,64 por mês	cerca de US$ 2,6
IP público Standard	cerca de US$ 0,005 por hora	cerca de US$ 3,7
Total aproximado		**cerca de US$ 13 por mês**

O preço do disco e o do IP foram consultados em outra região e podem diferir um pouco. Desalocar a VM interrompe a cobrança de computação (o disco e o IP continuam cobrando). O saldo de créditos do Azure for Students não é exibido pela linha de comando e deve ser consultado no Portal.

13. Conclusão

A migração para a CyberIncident API entregou uma aplicação com identidade própria, publicada na mesma infraestrutura Azure e validada de ponta a ponta: 55 testes automatizados locais, 30 verificações na URL pública, persistência confirmada após o reinício do serviço e a proteção de escrita do Nginx preservada. O banco SQLite recebeu a nova tabela sem perda da anterior, e o processo de atualização passou a abortar caso o backup falhe.

O trabalho permitiu aplicar conceitos centrais de computação em nuvem — máquina virtual, rede, firewall, disco persistente e implantação — e de desenvolvimento de APIs: modelagem, validação, documentação, testes e segurança. As principais limitações (ausência de HTTPS e de autenticação) estão documentadas, assim como os caminhos para superá-las.

14. Referências

- FastAPI. Documentação oficial. https://fastapi.tiangolo.com
- Pydantic. Documentação oficial. https://docs.pydantic.dev
- SQLAlchemy. Documentação oficial. https://docs.sqlalchemy.org
- SQLite. Documentação oficial. https://www.sqlite.org/docs.html
- Uvicorn. Documentação oficial. https://www.uvicorn.org
- Nginx. Documentação oficial. https://nginx.org/en/docs/
- systemd. Manual de referência. https://www.freedesktop.org/software/systemd/man/
- Microsoft. Tamanhos de máquinas virtuais no Azure. https://learn.microsoft.com/azure/virtual-machines/sizes
- Microsoft. Visão geral dos discos gerenciados do Azure. https://learn.microsoft.com/azure/virtual-machines/managed-disks-overview
- Microsoft. Grupos de segurança de rede do Azure. https://learn.microsoft.com/azure/virtual-network/network-security-groups-overview
- Microsoft. Calculadora de preços do Azure. https://azure.microsoft.com/pricing/calculator/
- IETF. RFC 5737 — Blocos de endereços IPv4 reservados para documentação. https://www.rfc-editor.org/rfc/rfc5737
- Swagger UI. https://swagger.io/tools/swagger-ui/